day 23 | 251022

note for those looking at this

I changed all of the y's to x's and all of the x's to t's. This is to avoid problems later on that we ran into later on in the Fall 2025. I also changed the dependence of our function for the differential equation so that the dependent variables (like `x_`) appear first and the independent variable (like `t_`) appears second. This again is to make things a little bit easier later on as well.

herein

Today we will continue to work with Euler's Method for solving differential equations, but this time we will turn to python to solve this problem now that we have some experience with Excel. We will talk about how to handle these first order differential equations that have only one variable on the right hand side, and then we will talk about how to handle them if they have two variables. These look like these two equations:

$$\frac{dx}{dt} = t^2 + t - 1 \tag{1}$$

$$\frac{dx}{dt} = x^3 + \sin t \tag{2}$$

Euler's Method is good, but the error grows over time as we have seen. This is part of every solution to differential equations, but it is something that we want to minimize. Next time, we will discuss some methods that will be better than Euler's method and won't take much longer to work through.

In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Or how about an equation that we have already solved with excel! Like this one:

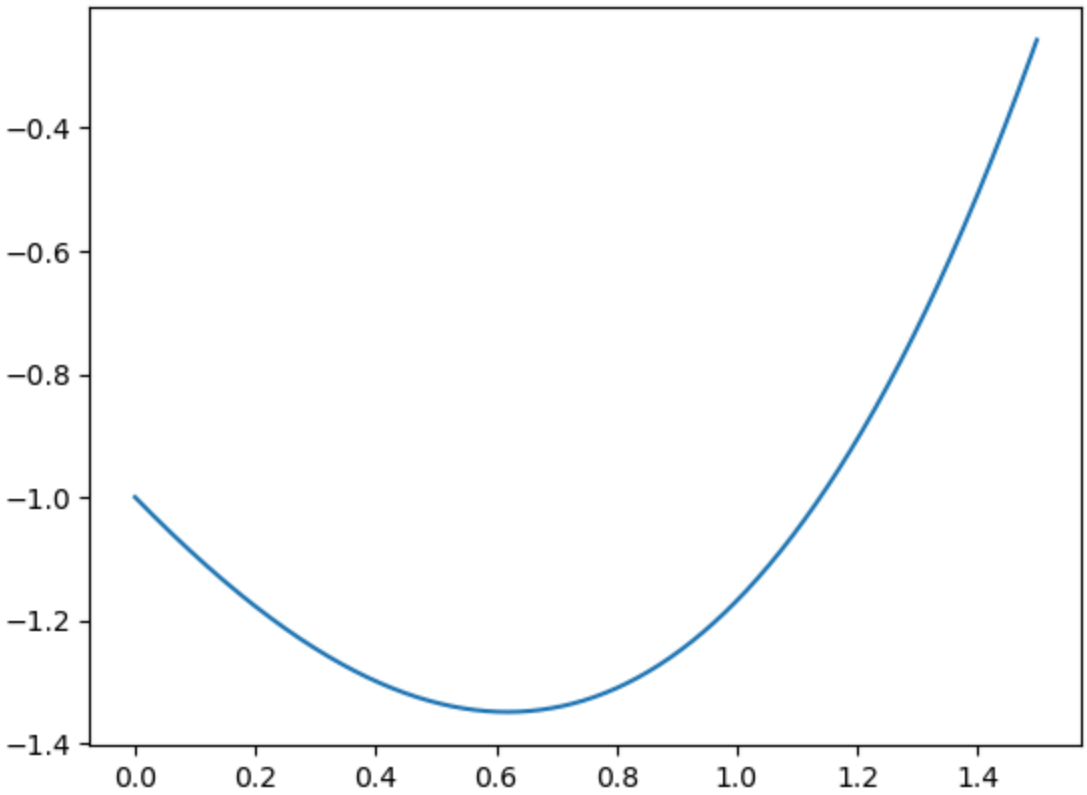
$$\frac{dx}{dt} = t^2 + t - 1$$

In [26]:
```
# define the differential equation
def f(x_, t_): # dx/dt
    return(t_**2+t_-1)

# define the boundary condition
a = 0.0
```

```
b = 1.5
N = 1000
dt = (b-a)/N

x = -1.0 # initial condition

tpoints = np.arange(a, b, dt)
xpoints = []

for i in tpoints:
    xpoints.append(x)
    x = x + f(x, i) * dt
```

In [27]:
```
fig1, ax1 = plt.subplots()
ax1.plot(tpoints, xpoints)
```

Out[27]: [<matplotlib.lines.Line2D at 0x74c3e0b8f590>]

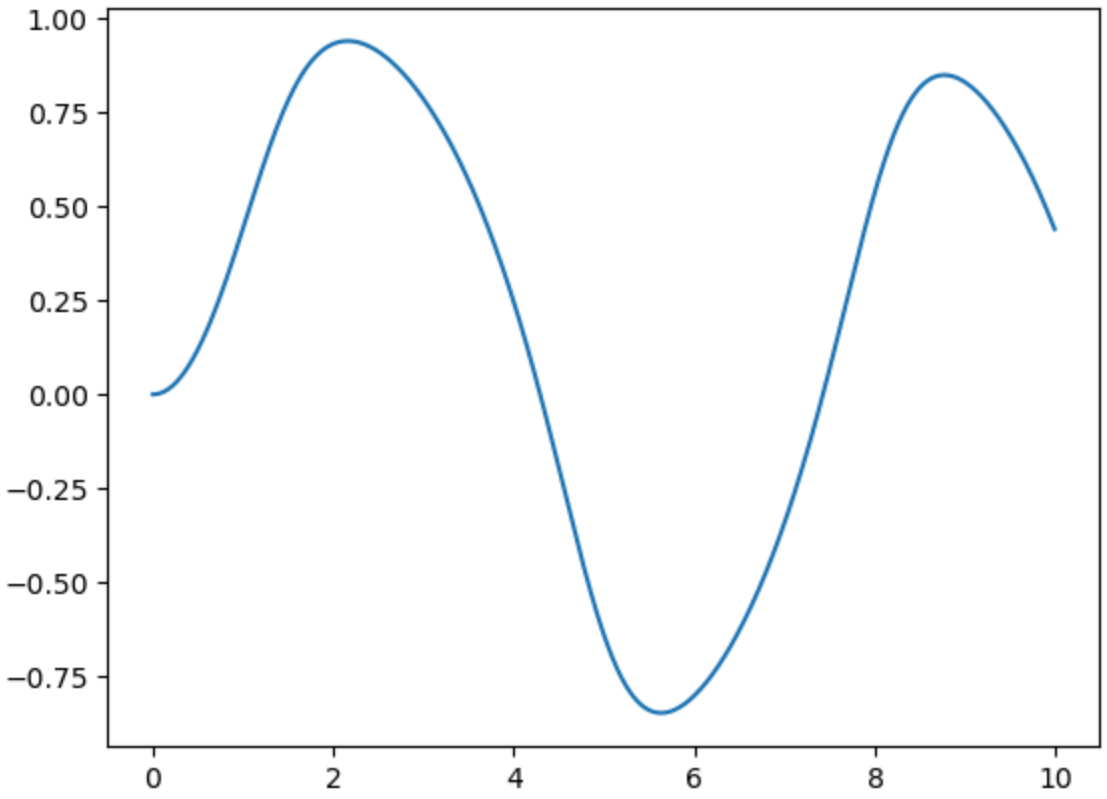


In [17]:
```
# define the differential equation
def f(x_, t_): # dx/dt
    return(-x_**3+np.sin(t_))

# define the boundary condition
a = 0.0
b = 10.0
N = 1000
dt = (b-a)/N

x = 0.0 # initial condition

tpoints = np.arange(a, b, dt)
```

```
xpoints = []

for i in tpoints:
    xpoints.append(x)
    x = x + f(x, i) * dt
```

In [18]:
```
fig0, ax0 = plt.subplots()
ax0.plot(tpoints, xpoints)
```

Out[18]:
```
[<matplotlib.lines.Line2D at 0x74c3df1d0140>]
```



In [ ]: